

Architecture de l'application

Pattern MVC (Symfony)

Le backend repose sur le pattern MVC implémenté avec Symfony.

Les **contrôleurs** reçoivent les requêtes HTTP envoyées par le frontend et exposent les endpoints de l'API.

La **logique métier** est déportée dans des services dédiés (gestion des matchs, participations, messages), ce qui permet de garder des contrôleurs simples et lisibles.

Les **modèles** correspondent aux entités Doctrine, représentant les données persistées en base.

Dans ce projet, Symfony est utilisé uniquement comme **API REST**, sans gestion de vues côté serveur. L'interface utilisateur est entièrement gérée par le frontend.

Architecture n-tiers

L'application suit une architecture **3-tiers avec séparation front/back** :

- **Client (frontend Vue.js)** : exécuté dans le navigateur, gère l'interface utilisateur
- **Serveur applicatif (Symfony)** : API REST, logique métier
- **Base de données (MySQL)** : stockage des données

Le frontend communique avec le backend via des requêtes HTTP (API REST, format JSON).

- Les **couches logiques** (frontend, backend, base de données) sont séparées conceptuellement
- Les **tiers physiques** peuvent être déployés sur une même machine ou sur plusieurs serveurs

Déploiement et conteneurisation

L'application est conteneurisée avec Docker et structurée en plusieurs services :

- un conteneur **frontend** (Vue.js)
- un conteneur **backend** (Symfony + PHP + serveur web Apache)
- un conteneur **base de données** (MySQL)

Ces conteneurs peuvent fonctionner sur un même hôte en développement, mais peuvent être séparés en production.

Organisation interne du backend

Le backend Symfony est organisé en plusieurs couches :

- **Controllers** : gestion des routes et des requêtes HTTP
- **Services** : logique métier
- **Repositories** : accès aux données via Doctrine
- **Entities** : représentation des données

Cette structure permet de respecter une séparation claire des responsabilités et de limiter les dépendances entre composants.

Bonnes pratiques et qualité du code

Le projet suit plusieurs principes de conception :

- **Single Responsibility Principle** : chaque classe a un rôle précis
- contrôleurs légers, logique métier centralisée dans les services
- utilisation de Doctrine pour abstraire l'accès à la base de données
- configuration externalisée via des variables d'environnement (.env)

Ces choix permettent de garantir un code maintenable et évolutif.

Composants externes et bibliothèques

Le projet utilise les technologies suivantes :

- Symfony (framework backend)
- Doctrine (ORM)
- MySQL (SGBD)
- Vue.js (frontend)
- Docker (environnement)

Des bibliothèques supplémentaires pourront être intégrées si nécessaire, notamment pour :

- l'authentification avec JWT
- la gestion d'API
- ou certaines fonctionnalités frontend

Cohérence avec la modélisation

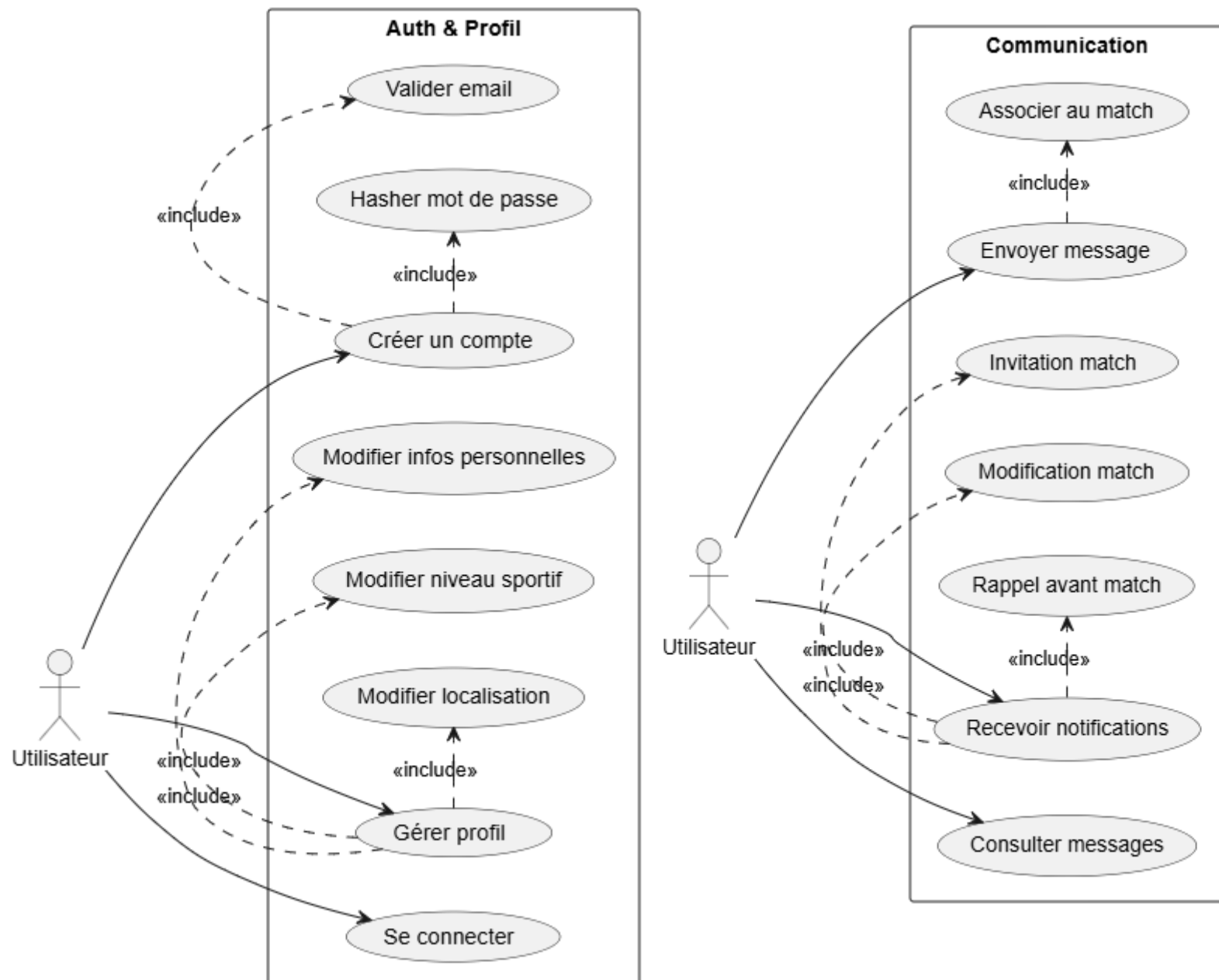
L'architecture est directement basée sur la modélisation réalisée au jalon précédent :

- chaque table correspond à une entité Doctrine
- les relations sont implémentées via les associations entre entités
- les services métier exploitent ces entités pour répondre aux cas d'utilisation

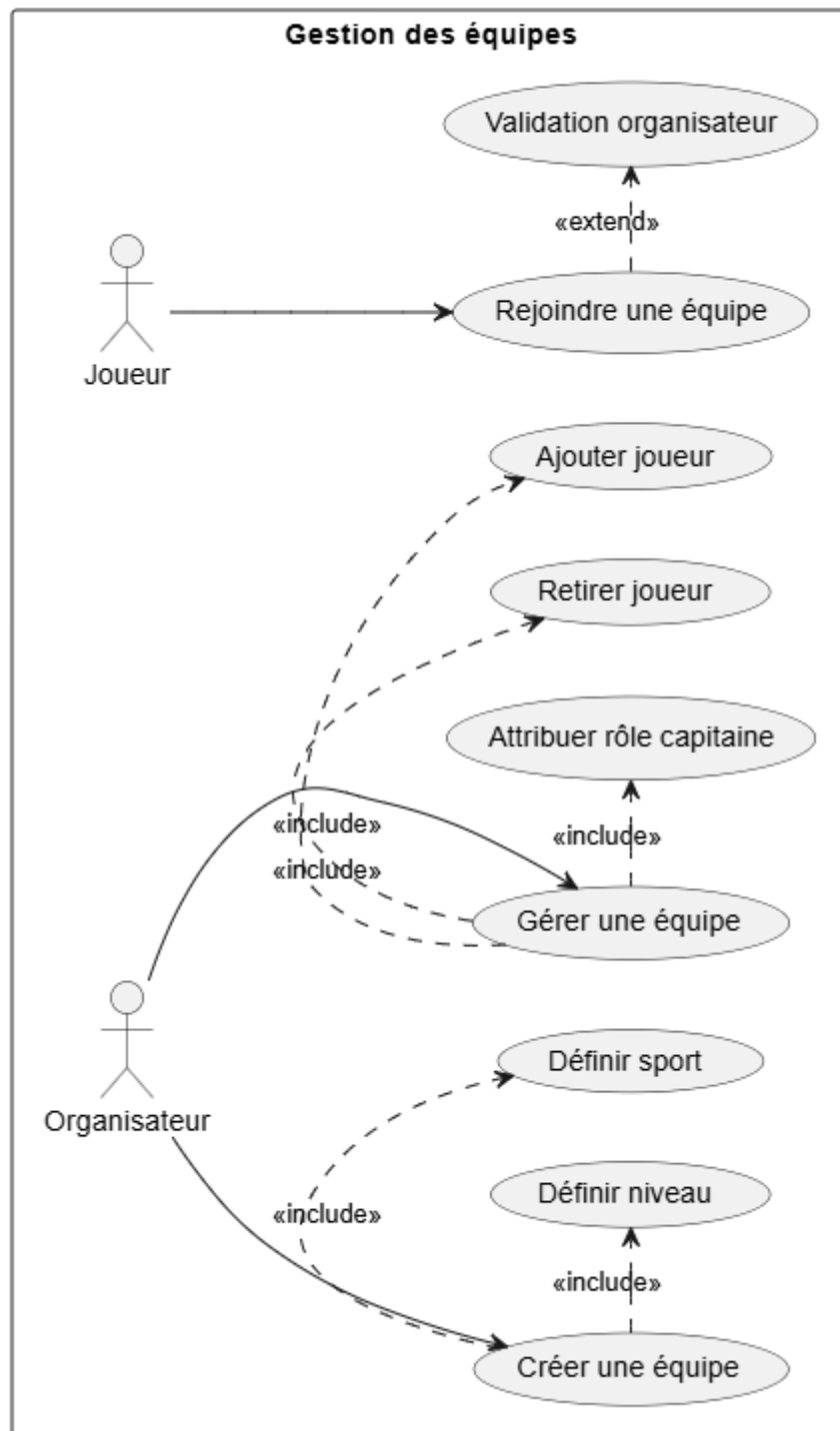
Cela garantit une cohérence globale entre la conception fonctionnelle, la base de données et l'implémentation technique.

Les différents diagrammes :

cas d'utilisations authentication et communication :



cas d'utilisations equipe :



cas d'utilisations match et résultat :

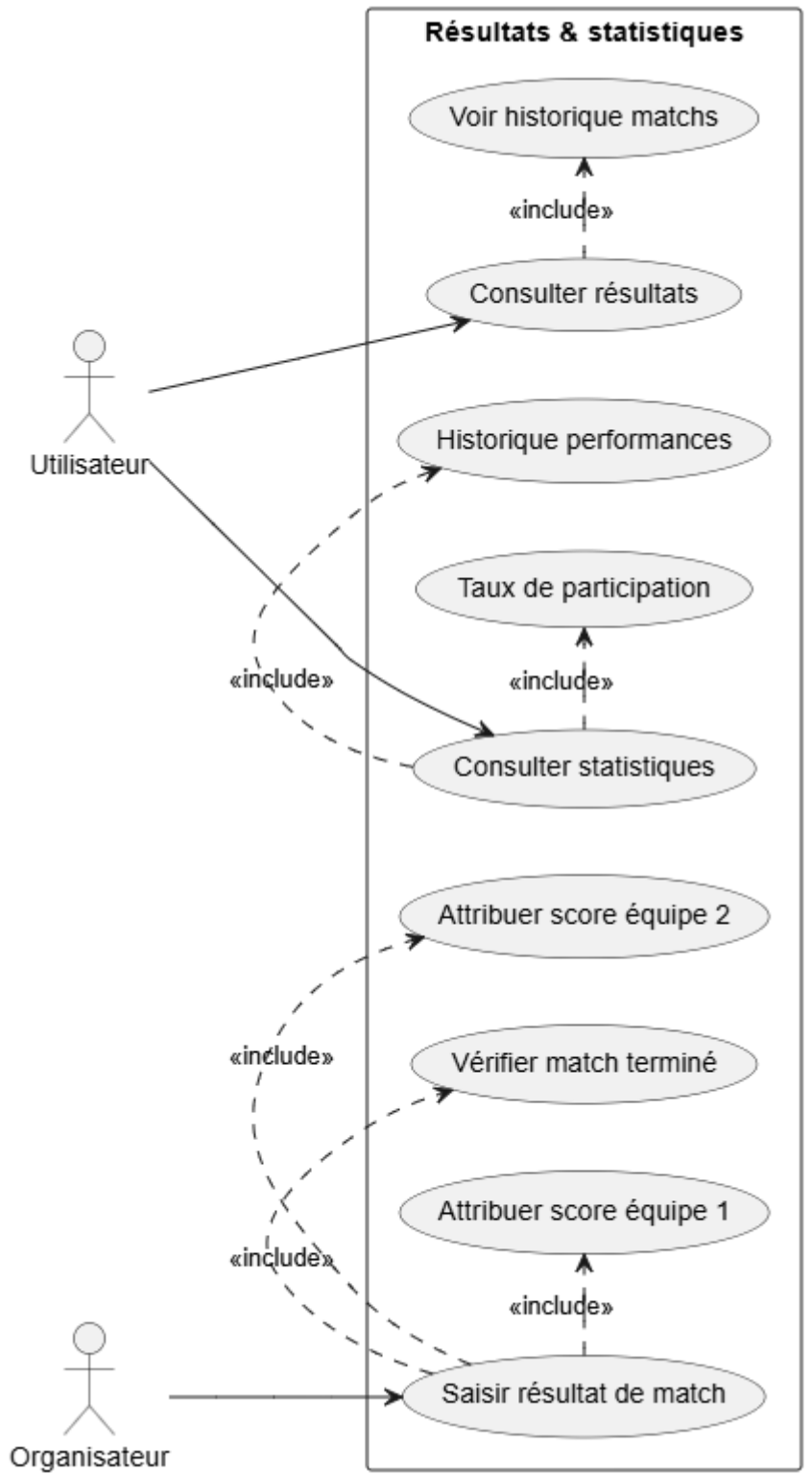
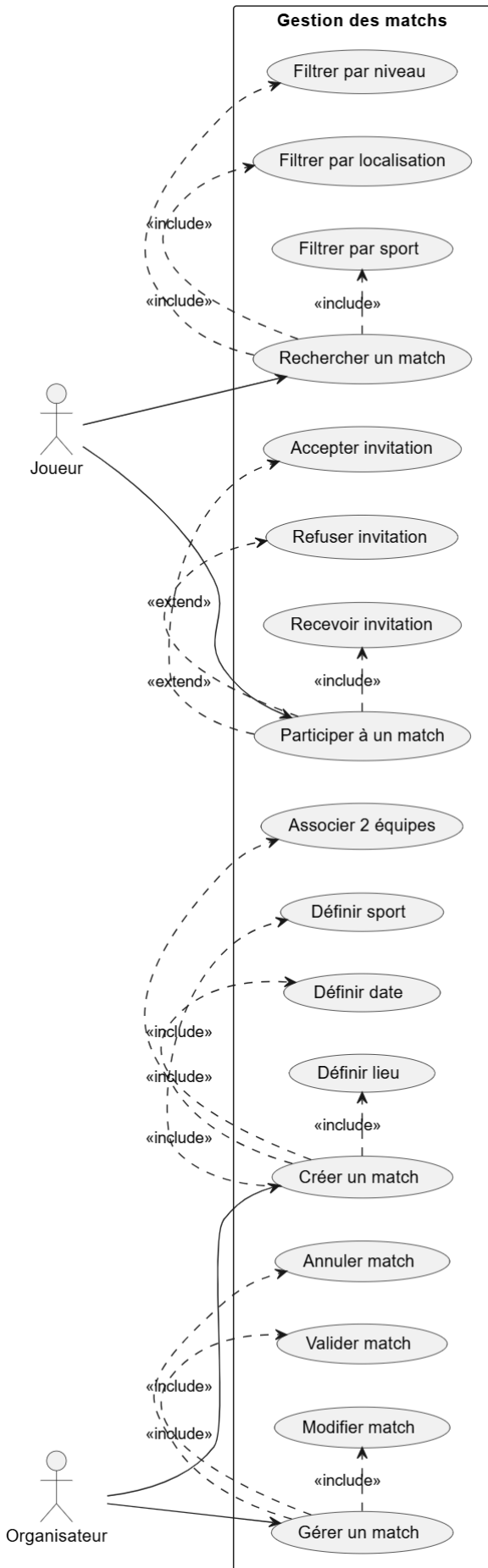


Diagramme de séquence créer un compte :

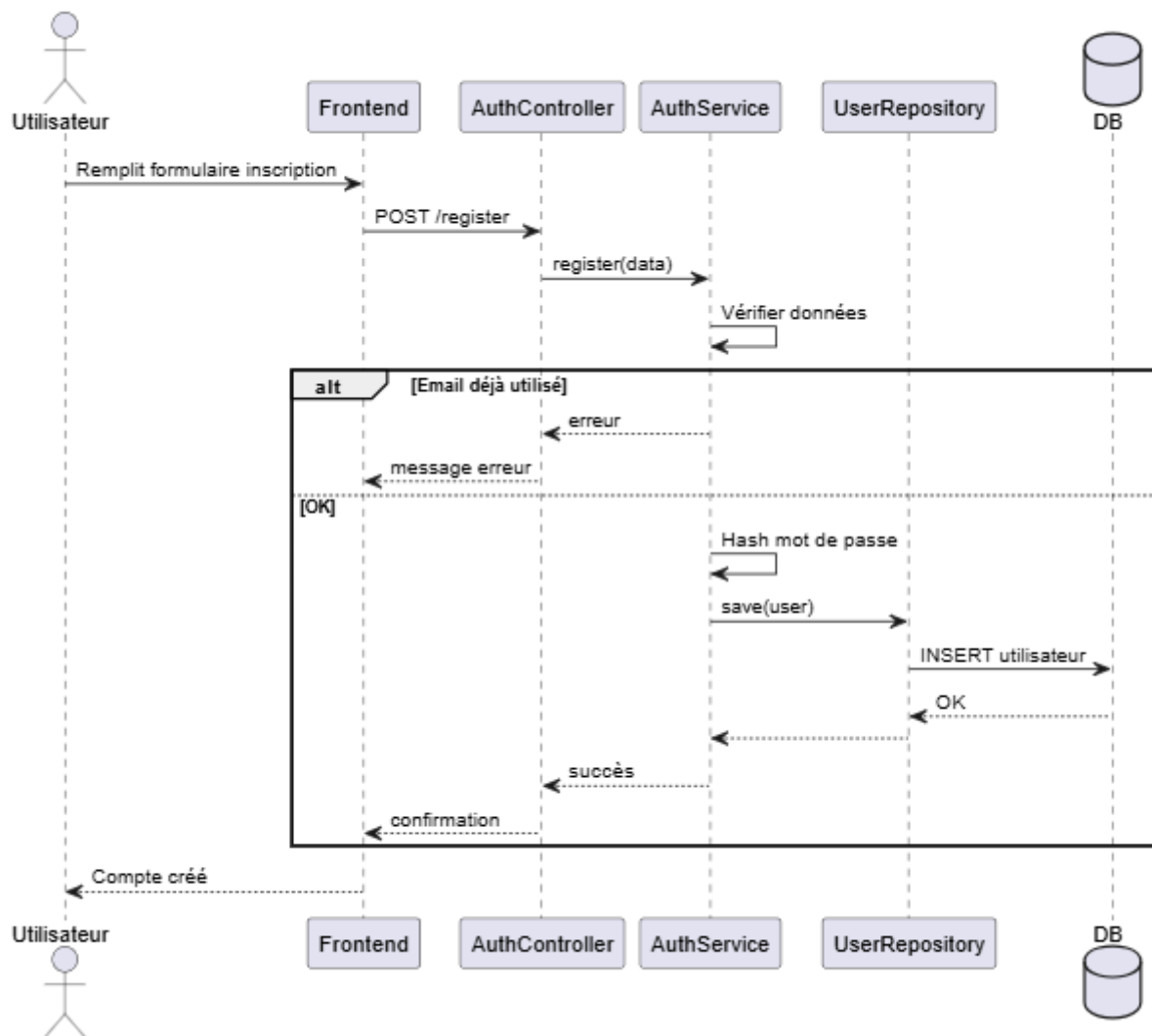


Diagramme de séquence créer un match :

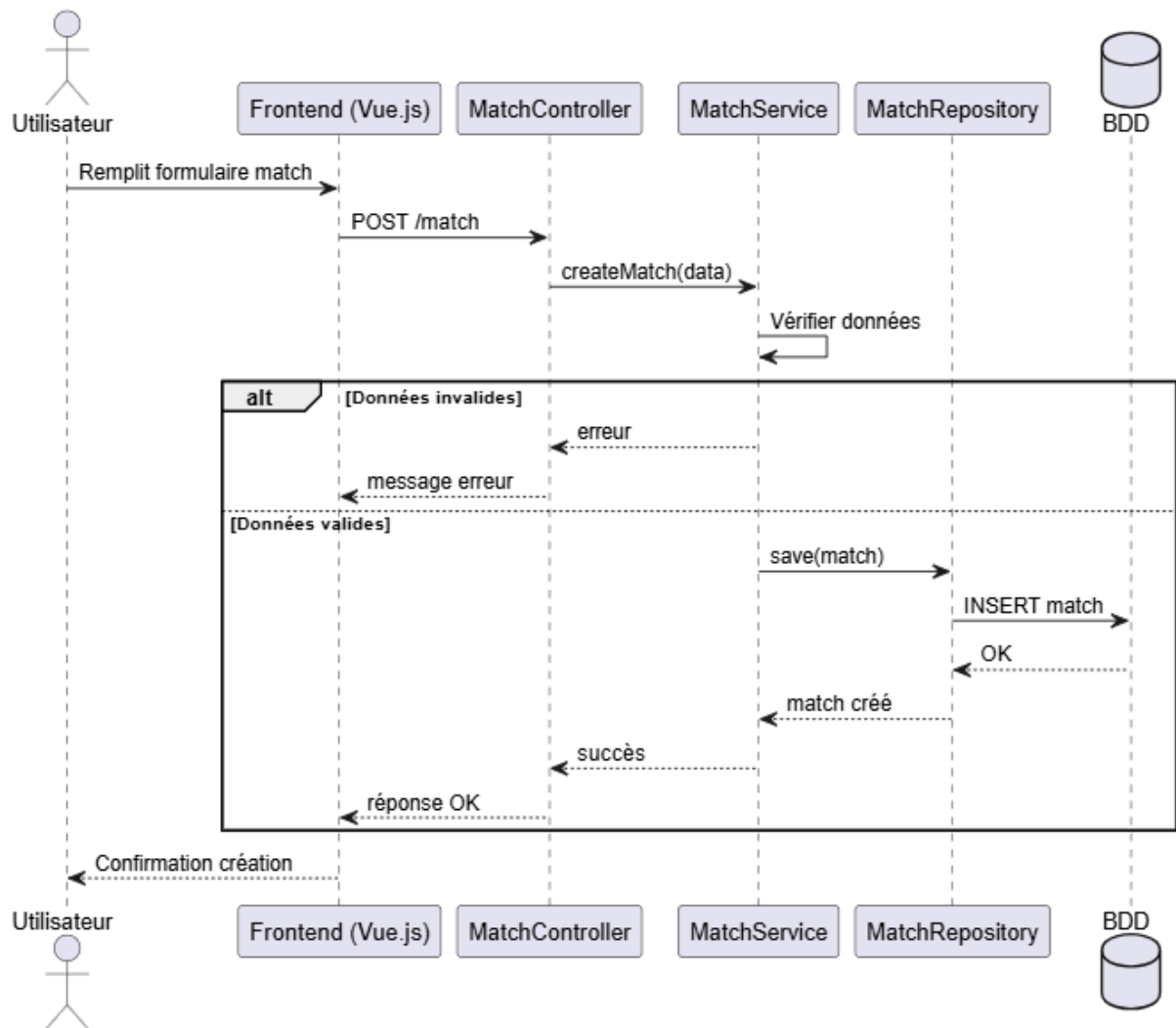


Diagramme de séquence participer à un match :

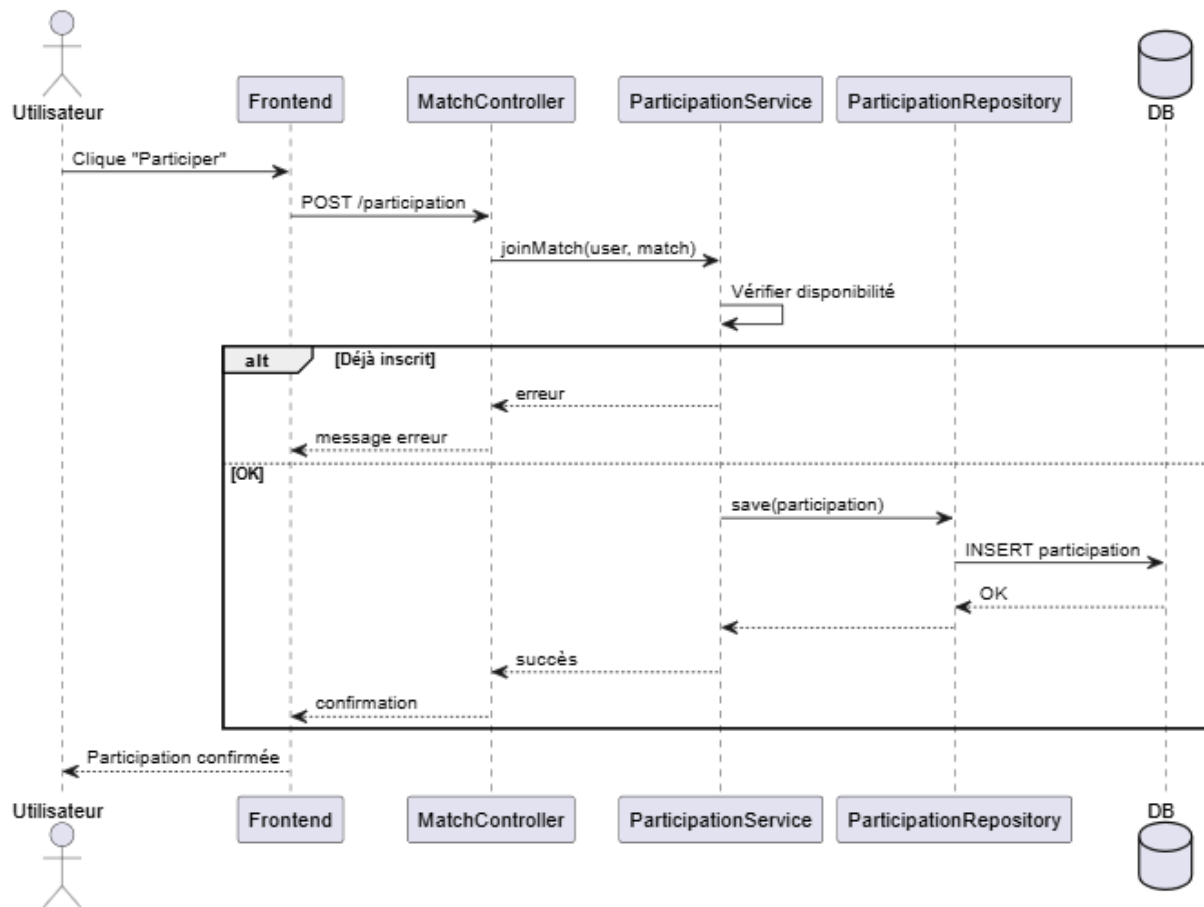
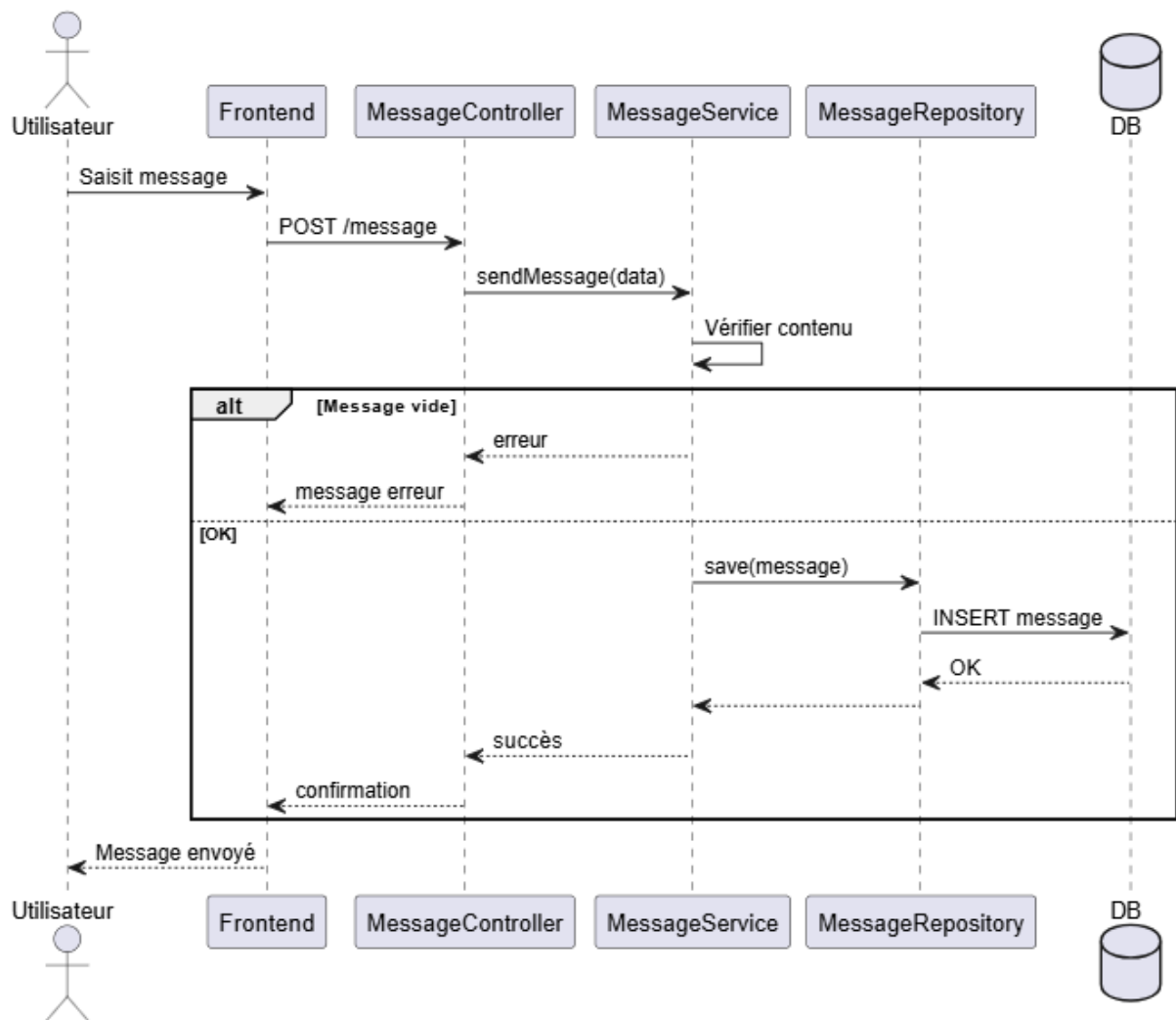


Diagramme de Séquence d'envoi de message :



Le diagramme de classe :

